Lab J: Final Arduino Lab! A Working Game.

Due: April 2, 9 p.m.

1 Introduction

In this homework assignment, you will be programming the classic game of Pong using Java and object-oriented programming principles. The project will see you implement a multi-class application with a GUI (graphical user interface) that integrates hardware from you Grove Board. You will be using the potentiometer to control the game's paddle. Ideally, the assignment will make you feel more confident navigating applications that consist of many Java classes and methods.

2 Programming Task

This lab may be slightly more complex than prior labs in that includes several modular class files, it is event-driven and contains a GUI, and it depends on hardware and various interfaces (like *IODeviceEventListener*, which you have seen in prior labs). However, the lab demonstrates some properties we might like to see in your Main and Final Projects. The projects you write should be similarly object-oriented and modular; you should define multiple classes with clear interfaces and data attributes that are well encapsulated. You should document your code similarly and submit a comparable set of unit tests.

Our Pong game has a class called 'Pong' which extends the JPanel class, so as to implement a simple game GUI (or graphical user interface). The Pong game contains instances of a Paddle and Ball; properties of these objects are defined in their own respective class files. The Main class serves as a game driver class and wraps itself around an instance of a Pong game. It then uses a Timer to repeatedly run the game's logic.

Before you start to write your own code, you will want to read all of the classes carefully so that you understand how they relate to one another and how the functionality of the game has been decomposed. Then, start your own work by assigning your NAME to the "name" parameter of the Main class.

Next, within the *Paddle* class, implement:

- The constructor *checkCollision(Ball b)*. This method will determine if the ball's position is at or within the boundaries of the paddle. If it is, return true (there's a collision!). Otherwise, return false.

In the *Ball* class, implement:

- The constructor *bounceOffWalls(int top, int bottom, int right)*. This method accepts the y positions of the top and bottom of the screen as well as the x position of the right hand side of the screen. It then checks to see if the ball is at or past the TOP or BOTTOM the screen and, if yes, it calls reverseY to reverse the Y direction of the ball. Alternatively, if the ball is at or past the RIGHT side of the screen, it calls reverseX to reverse the X direction of the ball.

Finally, in the *Pong* class, put all your pieces together and implement:

- The method *runGameCycle*(). This methods executes the game logic, and will be called repeatedly by the Timer in the Main class. Each time it is called it should move the ball on the screen (using the moveBall method). It should then call bounceOffWalls to check to see if the ball bounces off the wall and it should move the game paddle toward the position stored in paddlePosition. Next, it should check to see if the ball has collided with the paddle. If yes, the direction of the ball must change! Several methods in the Ball and Paddle class can be used to help you with these tasks. Finally, check to see if the ball is still in bounds at the end of each game cycle. If the ball is in bounds, return true (so the game can continue). But if the game leaves the board (i.e. if it's x position becomes negative), return false. This means the game is over!

- The event handler *onPinChange(IOEvent ioEvent)*. This method will tie your game paddle to the potentiometer on the Grove board. Within the method, first check to ensure the event has come from the potentiometer and not some other Pin. Assuming it has, get the value of the potentiometer map it onto a value that ranges between 0 to WINDOW_HEIGHT. Finally, assign your mapped value to the class attribute called 'paddlePosition'. The paddle should then move to this position when you play!

3 Testing your Classes

As always, a small number of test cases have been provided to help you test your implementations. Make sure you pass these tests! In addition, with this lab we ask that you submit either a video or a link to a short video that demonstrates your game-play (and your NAME on the GUI). This video SHOULD NOT EXCEED TEN SECONDS.

4 What to Submit

1. Pong.java, Main.java, Paddle.java and Ball.java
2. YourVideo.mp4 (or a URL to your video, online).

HAVE FUN AND GOOD LUCK!